Reflection: Context Engineering for TakeoutPick

HW3 · CS6983 AI Coding · Ruoyu Wang · Oct 30, 2026

How the rules file bridges the PRD and mockups to code

Our PRD and the Project 1 prototype were made in Claude Web. The IDE agent cannot open the prototype artifact. Its PRD is also long, and the agent does not know which parts are binding. `AGENTS.md` closes that gap in three ways. First, it **points** to the right sources and ranks them: `docs/PRD.md` §4 is "the only source for money math", `prisma/schema.prisma` means "never invent fields", and `docs/ui-spec.md` describes what the prototype established. Second, it **translates** visual decisions into rules that can be checked: totals before menu prices, "Healthiest" only when health levels differ, status always color plus a word, 44 px tap targets. Third, it **names the components** (`DealBadge`, `PriceWithDeal`, `MinOrderHint` …) and where they live, so code written today plugs into the screens of next week.

The before/after test surprised me. Without a rules file the agent still read the PRD and UI spec and met all 5 acceptance criteria. Good documentation already carries most of the *what*. The rules file added the *how*: the project-specific decisions a capable agent cannot infer. Examples are kebab-case component files, "time is a parameter", "only services touch the database" and "never mock Prisma". Those took rule violations from 6 to 0. The lesson for me is that a rules file should not repeat the PRD. It should hold the decisions that differ from what a typical developer would do.

What worked, what didn't, and what I changed

Worked: the version-trap section (Next 16, Prisma 7, zod 4) and the request-flow diagram. Neither rules run touched `@prisma/client` or put a clock call in domain code, and the v1.1 run reproduced the route → service → domain flow exactly. Linking every rule to a reason (in HTML comments) also helped. The agents quoted those reasons back, for example "Passed in so Server Components can render inside Suspense (AGENTS.md §1.2)".

Didn't work: two rules that are each sensible combined into a bad result. "Don't add what the issue doesn't ask for" plus "integration tests need a real database", with no GitHub access, made the v1.0 agent quietly skip the API criterion. A third rule, "contrast ≥ 4.5 : 1", could not be followed, because our own color tokens failed it and the agent had no way to measure. Rules that state a goal without a way to check it produce false compliance.

Changed: v1.1 makes scope explicit (every AC of the issue, PRD §5 as the fallback, never drop one silently). It says how to work around missing infrastructure ("written, not run") and makes `now` a required parameter. It also pre-approves the preferred libraries with an offline fallback, darkens the tokens and lists verified color pairs. The re-run met every criterion with no violations, and it exposed one more conflict ("route owns the clock" vs "never the real clock in tests"), fixed in v1.2. Each change is logged in the file's changelog with the evidence that triggered it. Rules are now reviewed in PRs like code.

How the GitHub Scrum setup organizes AI-assisted development

The issues are written as the agent's work orders. Each one is a user story slice with an acceptance-criteria checklist that maps to one test per box. That is why I could score the runs objectively. The issue number then threads through everything the agent produces: branch `feature/5-deal-status`, commits `test(#5)` → `feat(#5)`, and `Closes #5` in the PR. A reviewer can trace any AI-written line back to a story. Milestones and the board limit work in progress. Sprint 1 issues sit in Sprint Todo, so we never ask an agent to build Sprint 2 features early. The "In Review" column makes the human review step visible, because the author never self-merges AI code. Size labels and story points (25 for Sprint 1) let us check whether AI help really speeds us up across sprints.

What I would add or change for Sprint 2

- **Path-scoped rules.** `AGENTS.md` is now 370 lines and grows with each sprint. I would move feature-specific rules into nested `AGENTS.md` files (e.g. `src/lib/server/services/AGENTS.md`, `src/components/rooms/AGENTS.md` for Pusher). The root file would keep only what applies everywhere.
- **Executable rules.** Turn the checks from `measure.sh` into ESLint rules and a CI job (kebab-case filenames, no `db` import outside services, no `new Date()` in `lib/domain`, a contrast test for token pairs). A failing build teaches the agent more reliably than a sentence it may skip.
- **Issue-level AI notes.** Add an "AI context" section to the issue template: the files to read and the trickiest edge case. For real-time Group Pick (#9) that is the tie-break rule and the snapshot event format.
- **Repeat the experiment with three runs per condition** on a Sprint 2 issue, so the comparison measures the rules and not one run's luck. Also adopt A's good idea as a rule: screen-reader text for struck-through prices.